

Laboratorium 6: Kompresja danych – proste metody / Kodowanie Huffmana

Opracowanie zestawu zadań (część teoretyczna)

Kacper Bednarczuk

Wprowadzenie

W ramach tego laboratorium przeprowadzono praktyczne porównanie dwóch fundamentalnych algorytmów kompresji bezstratnej: statystycznego Kodowania Huffmana oraz sekwencyjnego Kodowania RLE ze znacznikiem. Poniżej przedstawiono formalne zasady działania obu metod oraz szczegółową analizę obliczeniową.

Kodowanie Huffmana (Statystyczne)

Zasada działania:

Jest to metoda kompresji oparta na entropii, która przypisuje krótsze kody symbolom o wyższej częstości występowania (prawdopodobieństwie), a dłuższe kody symbolom rzadziej występującym. Algorytm buduje drzewo, w którym częstość symboli decyduje o ich **głębokości**, a tym samym o **długości kodu**.

Formalna definicja (średnia długość kodu):

Długość kodu binarnego dla danego symbolu s_i jest wprost proporcjonalna do głębokości symbolu w drzewie. Średnia długość kodu L_{avg} jest minimalizowana przez sumę iloczynów prawdopodobieństw p_i i długości kodów l_i :

$$L_{avg} = \sum_{i=1}^k p_i \cdot l_i$$

Kluczowa cecha:

Algorytm **nie** uwzględnia położenia symboli. Wynik kompresji zależy tylko od ogólnej **statystyki** całego pliku.

Kodowanie RLE ze Znacznikiem (Sekwencyjne)

Zasada działania:

RLE (Run-Length Encoding) zastępuje długie serie identycznych znaków pojedynczą informacją o znaku i liczbie jego powtórzeń. W wariantcie **ze znacznikiem** system rozróżnia dane kompresowalne (seria ≥ 3 znaków) i niekompresowalne (literały), co pozwala **zapobiegać ekspansji** danych.

Struktura kodowania i koszty:

- **Seria:** Kodowana jako 24 bity (ZNACZNIK + LICZNIK + ZNAK).
- **Literał:** Zapisywany dosłownie (8 bitów na znak).

Kluczowa cecha:

Algorytm jest bardzo efektywny, gdy w danych występuje duża **korelacja przestrzenna** (wiele identycznych symboli występuje obok siebie), co jest jego przewagą w przypadku na przykład prostych obrazów.

Zadanie 1.1: Kodowanie Huffmana

1. **Zbiór danych A:** BABA_JAGA_BABA (14 znaków)

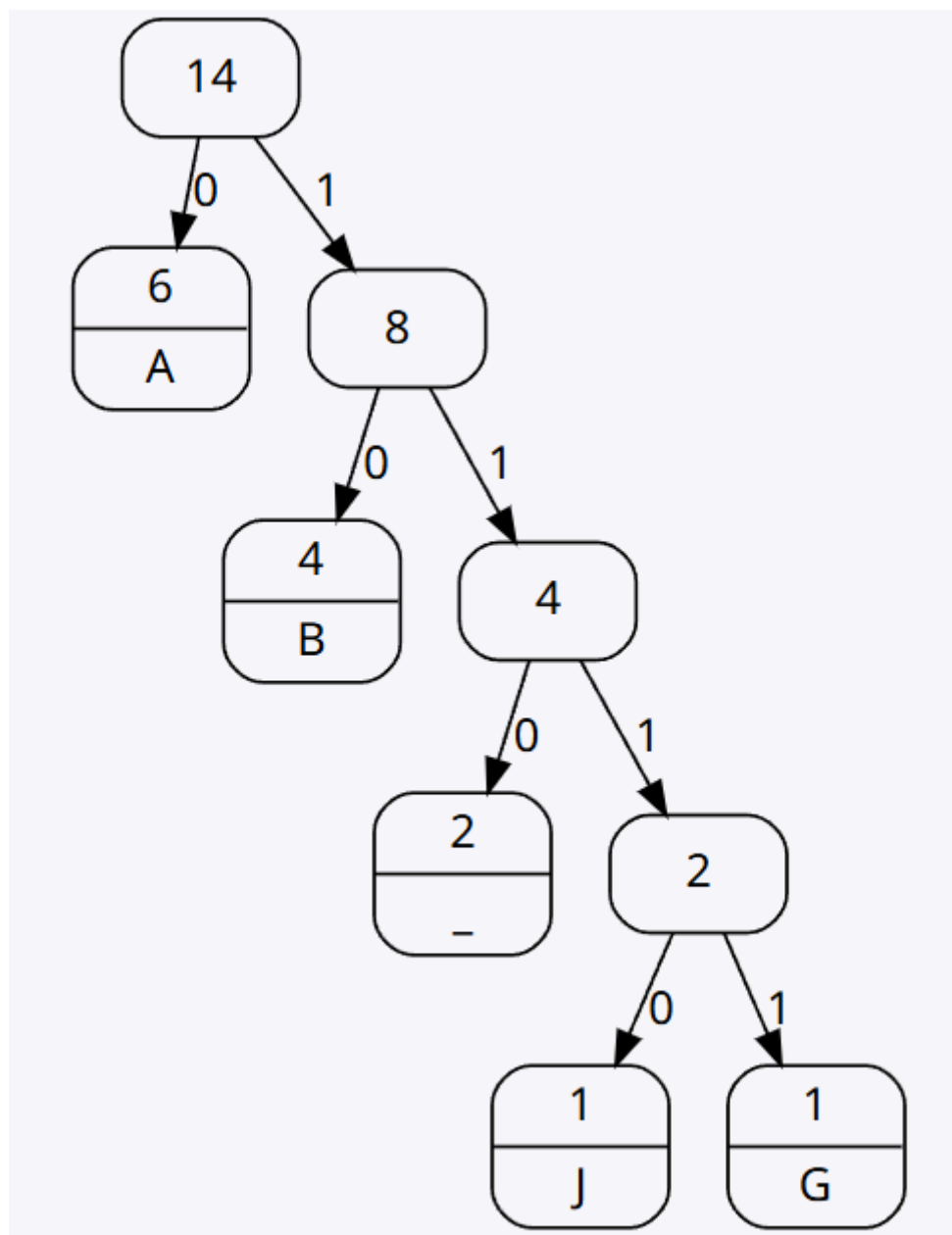
2. **Częstość wystąpień symboli:**

- A : 6
- B : 4
- _ : 2
- J : 1
- G : 1

3. **Drzewo Huffmana (kroki łączenia):**

- J (1) + G (1) $\rightarrow$ JG (2)
- _ (2) + JG (2) $\rightarrow$ _JG (4)
- B (4) + _JG (4) $\rightarrow$ B_JG (8)
- A (6) + B_JG (8) $\rightarrow$ AB_JG (14)

4. **Drzewo Huffmana (wizualizacja):**



Wizualizacja drzewa Huffmana dla Zbioru A. Zgodnie z diagramem, kody są przypisane wg zasady: lewa gałąź (0), prawa gałąź (1).

5. Tabela kodów Huffmana (Na podstawie wizualizacji):

Symbol	Częstość	Długość Kodu	Kod Binarny (Na podstawie wizualizacji)
A	6	1 bit	0
B	4	2 bity	10
–	2	3 bity	110
J	1	4 bity	1110
G	1	4 bity	1111

6. Obliczenia rozmiaru:

- Rozmiar oryginalny:** 14 znaków · 8 bitów/znak = **112 bitów**

- **Rozmiar po kompresji:** $6 \cdot 1 + 4 \cdot 2 + 2 \cdot 3 + 1 \cdot 4 + 1 \cdot 4 = 6 + 8 + 6 + 4 + 4 = 28$ bitów
 - **Stopień kompresji (oszczędność miejsca):** $100\% - (28/112) = 100\% - 25\% = 75\%$
-

Zadanie 1.2: Kodowanie RLE ze znacznikiem serii

1. **Zbiór danych B:** AAAAAAAAAABBBBBBBBBBBBBBBBBBCCCCC (32 znaki)

2. **Kodowanie RLE ze znacznikiem:**

- AAAAAAAAAA (10 'A') → \$10A (seria)
- BBBBBBBBBBBBBBBB (16 'B') → \$16B (seria)
- CCCCCC (6 'C') → \$6C (seria)
- Wynik: \$10A\$16B\$6C

3. **Obliczenia rozmiaru:**

- **Rozmiar oryginalny:** 32 znaki · 8 bitów/znak = **256 bitów**
 - **Rozmiar po kompresji:** 3 serie · 24 bity/seria = **72 bity**
 - **Stopień kompresji (oszczędność miejsca):** $100\% - (72/256) = 100\% - 28.125\% = 71.875\%$
-

Zadanie 1.3: Analiza krzyżowa

RLE na zbiorze A (BABA_JAGA_BABA)

- **Rozmiar wynikowy:** Ciąg nie zawiera żadnej serii ≥ 3 znaków, więc wszystkie 14 znaków są kodowane jako literały. Rozmiar wynosi $14 \cdot 8 = 112$ bitów.
- **Wnioski:** Nie nastąpiła kompresja (112 bitów → 112 bitów). Dzięki mechanizmowi literałów nie nastąpiła też ekspansja danych, co jest kluczową zaletą tego wariantu RLE.

Huffman na zbiorze B (AAAAAAAAAABBBBBBBBBBBBBBBBBBCCCCC)

- **Częstości i kody:** B: 16 (1 bit), A: 10 (2 bity), C: 6 (2 bity).
 - **Rozmiar wynikowy:** $(16 \cdot 1) + (10 \cdot 2) + (6 \cdot 2) = 48$ bitów.
 - **Wnioski:** Huffman osiągnął mniejszy rozmiar (48 bitów) niż RLE (72 bity) na tym konkretnym przykładzie.
-

Zadanie 1.4: Wnioski i porównanie

Zestawienie Wyników

Rozmiary w bitach dla porównywanych zbiorów:

Zbiór Danych	Rozmiar Oryginalny (8 bitów/znak)	Rozmiar po RLE (ze znacznikiem)	Rozmiar po Huffmanie (bity)
Zbiór A (Tekst)	112 bitów	112 bitów	28 bitów
Zbiór B (Grafika)	256 bitów	72 bity	48 bitów

Odpowiedzi na pytania

Która metoda okazała się lepsza dla zbioru A? Dlaczego?

Lepsze okazało się **Kodowanie Huffmana** (28 bitów). Huffman jest z natury stworzony do kompresji tekstu, ponieważ jego efektywność opiera się na analizie **statystyki wystąpień** symboli. Wysoka częstość symbolu 'A' (6/14) została nagrodzona najkrótszym 1-bitowym kodem, co dało maksymalną redukcję.

Która metoda okazała się lepsza dla zbioru B? Dlaczego?

Lepsze okazało się **Kodowanie Huffmana** (48 bitów), chociaż w idealnych warunkach to RLE (72 bity) jest typowane dla danych graficznych. Huffman wygrał w tym małym przykładzie, ponieważ dysponował bardzo małym słownikiem (tylko 3 symbole) o silnie zróżnicowanej częstości, co pozwoliło mu na ekstremalnie krótkie kody (1 bit dla symbolu B). W przypadku dłuższych serii (np. 1000 powtórzeń) lub większego alfabetu RLE zyskuje przewagę.

Jaka jest kluczowa zaleta metody RLE ze znacznikiem (w porównaniu do prostego RLE), którą ujawniła próba na zbiorze A?

Kluczową zaletą jest **zapobieganie ekspansji danych**. Dzięki Regule 2, literały (np. w tekście BABA... bez powtórzeń) są zapisywane dosłownie, kosztem **8 bitów na znak**. W prostym RLE każdy znak byłby zapisywany jako para [LICZNIK] [ZNAK] (np. 16 bitów), co **podwoiłoby** rozmiar pliku. Wariant ze znacznikiem jest "bezpieczny".

Na czym polega słabość kodowania Huffmana w kontekście zbioru B? (Innymi słowy: co takiego „widzi” RLE, czego Huffman nie jest w stanie „zauważyć”?)

Słabość Huffmana polega na jego **"ślepcie" na korelację przestrzenną**. Huffman widzi tylko globalną statystykę (np. 16 liter B), ale nie jest w stanie dostrzec, że te 16 liter B stoi **obok siebie** w idealnym bloku. RLE natomiast działa **wyłącznie** na tej sekwencyjnej informacji, co czyni go optymalnym do kompresji jednolitych obszarów.